DApp Project

(Hardhat + Next.js + Sepolia)

To create a minimal full-stack **Decentralized Application (DApp)**:

- A **smart contract** in Solidity (compiled, tested, and deployed with **Hardhat -** A Node.js toolkit to **compile, test, debug, and deploy** smart contracts.).
- **Write** contracts in VS Code, and **use Hardhat** to build/test/deploy them.
- A **web frontend** in **Next.js (React)** that connects to **MetaMask** and calls a contract using **ethers v6 -** Lightweight library to talk to Ethereum from Node or the browser (read/write, sign txns)**.**
- Deployed to the **Sepolia** Ethereum **testnet** (no real money), and the web app hosted on **Vercel/Netlify** (free tier).

Here –

- **Node.js** Node.js is software that lets you run JavaScript outside the browser.
- **Next.js** is a React tool that runs on **Node.js**.
  - **Frontend:** Makes web pages (React components) that run in the **browser**.
  - **Optional backend:** Lets you add small **server functions** (on Node.js)
- **Vercel** and **Netlify** are services that **host your website** for you.
  - **Put your site online** with a public HTTPS link.
  - **Build your code** (Next.js) from GitHub automatically.
  - **Fast worldwide delivery** using a CDN (Content Delivery Network), which is a group of servers around the world that store copies of your site's files (HTML, JS, images) and serve them from the location closest to the user. Thus, it loads pages quickly.

Project 1 - e-KYC & Consent Vault

- Verify someone's identity once, store only a **hash** (fingerprint) of the document on-chain, and let the user **grant or remove consent** for a verifier to check validity.
- It avoids re-submitting the same KYC documents to many institutions and gives the user control over who can check their status. No PDFs or images go on the blockchain — only cryptographic proofs.
- For verification, Issuer stored a fingerprint (hash) of the document on-chain when they first verified the holder. Holder turns ON consent for the verifier's wallet address. Verifier selects the same document file (bit-for-bit identical) in the web app. The app computes the file's hash in the browser. Only the hash is sent to the smart contract. The contract compares the hash values.

Roles

a. **Issuer**: A trusted party that **performs KYC** and "issues" the credential. E**xamples:** Bank, government ID authority, university registrar, NGO/co-op admin.
b. **Holder:** the user/artisan
c. **Verifier:** A party that wants to **check validity** of the holder's KYC. **Examples:** Another bank/fintech, marketplace, lender, employer.

Acceptance test: When holder gives consent and document hash matches, isValid returns **true**. Otherwise **false**.

Data design

On-chain (public, small):

- hash (bytes32): keccak256 fingerprint of the document bytes.
- issuer (address): who issued the credential.
- issuedAt (uint64), valid (bool).
- consent[holder][verifier] (bool): holder's switch.

Off-chain (private, big):

- The actual file (PDF/image) — keep in your computer or on IPFS (Web3.Storage/Pinata). Only its **hash** goes on-chain.

What to do?

1. A **smart contract** (Solidity) that:
    a. Saves a fingerprint (**hash**) of a verified document for a person.
    b. Lets the person **turn consent on/off** for a verifier (e.g., a bank).
    c. Returns **true/false** when a verifier checks validity (only if consent is on).
2. A **web app** (Next.js + ethers v6) that:
    a. Has simple tabs for **Issuer**, **Holder**, and **Verifier**.

    b. Can **hash a file in the browser**, call the contract, and show results.
3. **Deployed** on Sepolia testnet, **hosted** on Vercel/Netlify (free).

Outcome: a working DApp link + contract address + Documentation + Presentation

Architecture (simple view)

Browser (Next.js + ethers v6) ↔ MetaMask ↔ RPC (Alchemy/Infura) ↔ Ethereum Sepolia
Files (optional) → **IPFS** (store CID; keep hash on-chain)

Step-by-step build

a. Create the project folders
b. Configure Hardhat for Sepolia
c. Write the contract (KycVault.sol)
d. Compile the contract
e. Deploy to Sepolia
f. Frontend: connect and call the contract
g. Host the frontend on Vercel/Netlify.
h. Share your project's GitHub repository link + live URL + contract address.

Project 2: Micro-Loan DApp

- Small loans for artisans using smart contracts: request → fund → withdraw → repay → handle default automatically. Clear, auditable rules; fewer middlemen.
- Let a **borrower** request a small loan. A **lender** funds it. The contract **escrows** the money until the borrower withdraws. Borrower **repays** by a deadline; otherwise the loan becomes **defaulted**. All rules are in the smart contract; every step is transparent.
- **Testnet only (Sepolia). No real money. Use dummy amounts.**

Roles

a. **Borrower** – asks for a loan with principal, total repayment, and due date.
b. **Lender** – funds a specific loan (escrowed in the contract).
c. **(Optional) Admin** – can pause the contract or set limits (nice-to-have, not required).

Data design

On-chain (public, small):

- Loan struct per ID:
  - borrower (address)
  - lender (address) (empty until funded)
  - principal (uint256) – amount to be funded
  - repayment (uint256) – principal + flat fee (keep simple for V1)
  - dueDate (uint64) – UNIX time
  - status (enum) – Requested/Funded/Withdrawn/Repaid/Defaulted
- **Events:** Requested, Funded, Withdrawn, Repaid, Defaulted

What to do?

1. **Smart contract (Solidity)** that:
   a. Creates a loan request (requestLoan(principal, repayment, dueDate)).
   b. Accepts funding (fund(id) with **exact** principal, escrowed).
   c. Lets the borrower withdraw once funded (withdrawToBorrower(id)).
   d. Accepts **exact** repayment before due date (repay(id)), pays lender, marks **Repaid**.
   e. After due date (if not repaid), allow marking **Defaulted** (markDefault(id)).
   f. Emits events for each state change.
2. **Web app (Next.js + ethers v6)** that:
   a. Has tabs or sections: **Borrower** and **Lender**.
   b. Borrower: create request; see status; withdraw; repay.
   c. Lender: list **Requested** loans; fund a chosen loan; see portfolio/status.
3. **Deploy** on **Sepolia** and **host** on **Vercel/Netlify** (free).

Outcome: a working DApp link + contract address + **Documentation** + **Presentation**.

Architecture (simple view)

Browser (Next.js + ethers v6) ↔ MetaMask ↔ RPC (Alchemy/Infura) ↔ Ethereum Sepolia

Step-by-step build

a. **Create the project folders**.

b. **Configure Hardhat for Sepolia** (SEPOLIA_RPC_URL, PRIVATE_KEY in contracts/.env).

c. **Write the contract** MicroLoan.sol.

d. **Compile** the contract

e. **Deploy** to Sepolia and copy the **address**.

f. **Frontend: connect and call the contract**

g. **Host** on Vercel/Netlify and deploy.

h. **Submit** Git Hub repository URL + live URL + contract address + short demo video.

Project – 3: Craft-Chain Traceability (ERC-721 batches)

- Each product/batch gets a unique on-chain ID (NFT). Every hand-off (artisan → co-op → retailer → buyer) is recorded, giving end-to-end transparency.
- Keep big/Private data **off-chain** (e.g., IPFS); put **fingerprints + minimal facts** on-chain.
- **Testnet only (Sepolia)**. No real money or PII.

Simple flow:
Mint batch → Record steps → Transfer custody → Show timeline

Roles

a. **Artisan / Producer** – creates (mints) a batch NFT and sets initial metadata.
b. **Co-op / Distributor / Retailer** – takes custody (transfer) and records processing/transport steps.
c. **Buyer** – final holder; can view the full chain of custody via token ID (QR).

Data design

On-chain (public, small):

- **ERC-721 token** per batch (tokenId).

Off-chain (private/large):

- **Metadata JSON** on IPFS (image, description, attributes like origin, material, dates).

What to do?

1. **Smart contract (Solidity)** that:

    a. Use a ready-made NFT standard (ERC-721) from OpenZeppelin so you don't reinvent the wheel.
    b. Add a role for who can create batches (MINTER). Only those addresses can mint.
    c. emits StepRecorded (actor = msg.sender).
    d. Uses standard ERC-721 transfers for custody (no custom function needed).

2. **Web app (Next.js + ethers v6)** that:
    a. **Mint tab** (Artisan/Admin): upload metadata to IPFS → get **CID**.
    b. **Record Step tab** (current owner): add a step (create a small JSON → pin to IPFS → compute hash.
    c. **View Batch tab**: enter/scan **tokenId** (QR) → show **owner**, tokenURI data (from IPFS), **Transfer** history, and **StepRecorded** events as a **timeline**.

3. **Deploy** on **Sepolia**, **host** on **Vercel/Netlify** (free).

Outcome: working DApp link + contract address + **Documentation** + **Presentation**.

Architecture

Browser (Next.js + ethers v6) ↔ MetaMask ↔ RPC (Alchemy/Infura) ↔ Ethereum Sepolia
Files & metadata → **IPFS** (store **CID**; keep hashes on-chain)

Step-by-step build

a. **Create the project folders**.

b. **Configure Hardhat for Sepolia**

c. **Write the contract** CraftBatch721.sol.

d. **Compile**

e. **Deploy** to Sepolia and copy the **address**.

f. **Frontend**.

g. **Host** on Vercel/Netlify; set env var; deploy.

h. **Submit** repo + live URL + contract address + short demo video.

1) Prerequisites (one-time installs)

Software

1. **Node.js LTS (version - 18 or 20)** — runs build tools (comes with **npm - node package manager to download and manage JavaScript libraries**).

   [Go to the official Node.js website → Click the **LTS** button (green) → Pick **22.x** → Run the installer → keep defaults → finish. Open **PowerShell** and check: **node –v** and **npm –v]**

2. **Git** — the tool on a computer (version control).

   [Go to the official site (git-scm.com) → download **"Git for Windows"**. Run the installer: **Components:** keep defaults (Git Bash + Git GUI + Credential Manager). **Editor:** choose VS Code if you use it. **PATH:** "**Git from the command line and also from 3rd-party software**". **HTTPS transport:** Use **OpenSSL**. **Line endings: Checkout Windows-style, commit Unix-style (recommended)**.]

3. **VS Code** — editor to write and edit smart contracts (Solidity) and website code (Next.js/React) (install extensions: *Solidity* by Juan Blanco, *Prettier*)

   [In the powershell, type - winget install --id Microsoft.VisualStudioCode –e and press Enter]

4. **MetaMask** — browser wallet (Chrome/Edge/Brave/Firefox)

   [Open the **Chrome Web Store** and search **MetaMask** → Verify the publisher is **"MetaMask"** (official) and the listing has lots of users/reviews → Click **Add to Chrome** → **Add extension**. Click the puzzle icon → **Pin** MetaMask so it's visible.]

   [Choose **Create a new wallet** (or **Import** if you already have a wallet). Set a strong **local password** (this only locks/unlocks the extension on this device). **Secret Recovery Phrase (12 words):** Write it **offline** (paper/steel), **Do not** store in screenshots/cloud/email/WhatsApp. **Never share it**—not with "support," not with anyone. It recovers your funds. Confirm the phrase and finish onboarding. **Auto-lock**: Settings → Security & Privacy → set a short auto-lock time.]

Free Accounts

5. **Alchemy *or* Infura** — gives your tools a **connection to** Sepolia. sign up to create a Sepolia endpoint, and copy the HTTPS URL.

   [Sign up and open the **Alchemy Dashboard**. **Create App / API Key** → pick chain (e.g., Ethereum), network (e.g., Sepolia). Copy curl.]

   [Open **Windows PowerShell**. Paste these 4 lines (replace `YOUR_API_KEY` first):

```
        $RPC = "https://eth-sepolia.g.alchemy.com/v2/YOUR_API_KEY"
        $headers = @{ 'Content-Type' = 'application/json' }
$body = '{"jsonrpc":"2.0","id":1,"method":"eth_blockNumber","params":[]}'
        Invoke-RestMethod -Uri $RPC -Method Post -Headers $headers -Body $body
```

You should see a JSON with a `"result"` like `0xABCDEF`]

6. **Sepolia faucet** — free test ETH